

Aula 10 | Consumo de APIs Reais



Objetivos da aula de hoje

Ao final, você deve conseguir:

- Entender o que é uma API REST e como ela funciona. Usar `fetch()` para consumir APIs brasileiras usadas no contexto de ERP e negócios. Consultar CNPJ, cotação de moeda, feriados e municípios – dados que o SAP consome e processa diariamente.

Problema gerador

Como conectar seu sistema ao mundo real?

Pergunta guia

Todo formulário de cadastro brasileiro pede CEP. Em vez de o usuário preencher rua, bairro e cidade manualmente, o sistema busca tudo automaticamente. Como construir isso?

- Estudo de caso 1: Auto-preenchimento de endereço por CEP (ViaCEP)
- Estudo de caso 2: Sistema de tarefas – listar e criar via API (JSONPlaceholder)
- Expositiva: `fetch()` na prática – GET e POST
- Prático: Cotação, meteorologia e CNPJ (AwesomeAPI, Open-Meteo, BrasilAPI)



Conceitos

Conceitos-chave

- API REST – contrato entre sistemas: URL + verbo HTTP + JSON
- Endpoint – URL que representa um recurso (/cnpj/v1/000000000000191)
- GET – buscar dados | POST – criar | PUT/PATCH – atualizar | DELETE
- Status code – 2xx ok | 4xx erro do cliente | 5xx erro do servidor
- JSON – formato padrão de troca entre sistemas (o SAP exporta e importa JSON)
- fetch() – função nativa do browser para requisitar APIs
- await res.json() – lê o corpo da resposta como objeto JavaScript



O que é uma API REST?

API REST:

REST = padrão arquitetural: URL identifica o recurso, verbo identifica a ação. O servidor responde com JSON — o mesmo formato que você já usa em objetos JS. O SAP expõe e consome APIs REST: o SAP Business Technology Platform (BTP) usa exatamente esse padrão para integrações entre sistemas.

// O SAP consome APIs o tempo todo internamente.

// Você pode fazer o mesmo nos seus sistemas.

// Estrutura de uma requisição REST:

// VERBO	+ URL (endpoint)	→ AÇÃO
// GET	/cnpj/v1/000000000000191	→ buscar empresa
// GET	/feriados/v1/2025	→ buscar feriados
// POST	/pedidos	→ criar pedido
// PUT	/fornecedores/42	→ atualizar cadastro



fetch() na prática — buscando dados reais

fetch nunca rejeita em 404 ou 500 — só por falha de rede. Por isso `res.ok` é essencial: `true` apenas para status 200–299. Cole no browser agora e veja o JSON: `brasilapi.com.br/api/cnpj/v1/000000000000191`

`fetch()` é a função do browser para fazer requisições HTTP.
Ela retorna uma Promise — use `async/await` para esperar a resposta.

```
// Passo 1 — fazer a requisição (retorna Promise<Response>)  
const res = await fetch('https://brasilapi.com.br/api/cnpj/v1/000000000000191');
```

```
// Passo 2 — checar o status (res.ok = true para 200-299)
```

```
if (!res.ok) throw new Error(`Erro ${res.status}`);
```

```
// Passo 3 — ler o corpo como JSON
```

```
const empresa = await res.json();
```

```
console.log(empresa.razao_social); // 'BANCO DO BRASIL SA'  
console.log(empresa.município);    // 'BRASILIA'
```



Estudo de Caso 1: Auto-preenchimento de endereço (ViaCEP)

Acessem: <https://viacep.com.br/>

```
const res = await fetch(`https://viacep.com.br/ws/${limpo}/json/`);
```

```
if (!res.ok) throw new Error(`Erro HTTP ${res.status}`);
```

```
const end = await res.json();  
if (end.erro) throw new Error('CEP não encontrado');
```

Parâmetro de rota (path param): o CEP é parte da URL – /ws/01310100/json/ – não é query string.

A resposta tem logradouro, bairro, localidade, uf, ddd, ibge. GOTCHA: ViaCEP retorna status 200 mesmo para CEP inexistente, mas com { "erro": true } no corpo – cheque end.erro! CEPs para testar: 01310-100 (Av. Paulista SP), 93010-020 (São Leopoldo RS).

```
// Cenário: usuário digita o CEP → sistema preenche o resto
```

```
// Endpoint: GET https://viacep.com.br/ws/{CEP}/json/
```

```
// Parâmetros: CEP como parte da URL (path param), sem query string
```

```
async function buscarEnderecoPorCEP(cep) {  
  const cepFormatted = cep.replace(/\D/g, ''); // '01310-100' → '01310100'
```

```
  if (cepFormatted.length !== 8)  
    throw new RangeError('CEP deve ter 8 dígitos');
```



Estudo de Caso 2: Sistema de Tarefas (JSONPlaceholder)

Query param (?userId=1): filtra no servidor – mais eficiente que buscar tudo e filtrar no cliente.

Outros exemplos de query params em APIs reais: ?page=2&limit=10 (paginação), ?search=texto (busca), ?status=active (filtro).

[JSONPlaceholder](#) tem /posts, /todos, /users, /comments, /albums – perfeito para simular qualquer sistema CRUD.

Cenário: listar tarefas de um usuário e criar uma nova tarefa.

```
// Query param (?chave=valor): filtra recursos sem mudar o endpoint
const BASE = 'https://jsonplaceholder.typicode.com';
```

```
// GET com query param: busca só os todos do usuário 1
```

```
const res = await fetch(`${BASE}/todos?userId=1`);
const tarefas = await res.json(); // array de até 20 itens
```



POST — Enviando Dados para a API

POST é o verbo para CRIAR recursos: formulários, cadastros, pedidos.

- method: 'POST' — informa que estamos enviando dados
- headers: Content-Type — diz ao servidor que o body é JSON
- body: JSON.stringify() — converte objeto JS em string JSON

Exemplo — criar um pedido (JSONPlaceholder como simulador):

```
const res = await fetch('https://jsonplaceholder.typicode.com/posts', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ title: 'Pedido #1042', userId: 5 }),  
});
```

```
const criado = await res.json();  
console.log(criado.id); // 101 — o servidor retorna o recurso criado
```

Em APIs reais de ERP: um POST pode criar um pedido de compra, um pagamento ou um fornecedor. O servidor devolve o recurso com o ID gerado. PUT e PATCH atualizam. DELETE remove. Hoje praticamos GET e POST — os mais usados no dia a dia.



Exercícios de fixação

Grupo 1: “Conversor de Moedas”

AwesomeAPI – base: `economia.awesomeapi.com.br/last/{par}`

ex: `// GET /last/USD-BRL → { USDBRL: { bid, ask, pctChange, name } }`

1. Busque a cotação do USD-BRL e exiba: compra (bid), venda (ask) e variação do dia.
2. Busque USD-BRL e EUR-BRL em paralelo com ``Promise.all`` e exiba os dois.
3. Calcule quanto custaria comprar 100 dólares com a cotação atual.
4. Trate o erro para par inválido (ex: `'XYZ-BRL'`) com `try/catch`.
5. A chave da resposta une o par sem traço: `'USD-BRL' → `data['USDBRL']``. Mostre isso.
6. Busque também GBP-BRL e ARS-BRL e monte uma tabela de cotações.
7. ****Desafio:**** busque o histórico de 30 dias: ``GET /USD-BRL/30`` e exiba máximo e mínimo.

Feito é melhor que perfeito: comece simples, melhore depois!



Exercícios de fixação

Grupo 2: “Painel Meteorológico”

Open-Meteo – base: `api.open-meteo.com/v1/forecast` (sem cadastro)

ex:

```
// ?latitude=-29.68&longitude=-53.80&current=temperature_2m,windspeed_10m
```

1. Busque temperatura e vento atual de Santa Maria RS (lat -29.68, lon -53.80).
2. Exiba: temperatura (°C), velocidade do vento (km/h) e hora da medição.
3. Adicione `&daily=temperature_2m_max,temperature_2m_min`` para obter máx/mín dos próximos 7 dias.
4. Busque Porto Alegre, Caxias do Sul e São Leopoldo em paralelo com ``Promise.all``.
5. Descubra qual das 3 cidades está mais quente agora.
6. Trate erro de rede com try/catch e exiba mensagem amigável.
7. ****Desafio:**** crie uma função genérica ``buscarClima(lat, lon, campos)`` parametrizável.

Feito é melhor que perfeito: comece simples, melhore depois!



Exercícios de fixação

Grupo 3: “Consulta de CNPJ”

BrasilAPI – base: `brasilapi.com.br/api/cnpj/v1/{CNPJ}`

ex:

```
// GET /cnpj/v1/00000000000191 →  
// { cnpj, razao_social, municipio, uf,  
// descricao_situacao_cadastral, cnae_fiscal_descricao }  
async function buscarEmpresa(cnpj) {  
  // sua implementação  
}
```

1. Implemente ``buscarEmpresa(cnpj)`` que limpa o CNPJ e faz o fetch.
2. Exiba: razão social, município/UF, situação cadastral e atividade principal.
3. Trate o erro para CNPJ inválido (status 404 ou 422).
4. Teste com: 00000000000191 (Banco do Brasil), 00.360.305/0001-04 (Caixa).
5. Exiba a situação: se não for "ATIVA", destaque com um aviso.
6. Busque ambas em paralelo com ``Promise.all``.
7. ****Desafio:**** o CNPJ tem dígitos verificadores. Pesquise o algoritmo e valide antes do fetch.

Feito é melhor que perfeito: comece simples, melhore depois!



Correção

```
const BASE = 'https://economia.awesomeapi.com.br';
```

```
// 1) Cotação USD-BRL:  
const res = await fetch(`${BASE}/last/USD-BRL`);  
if (!res.ok) throw new Error(`Erro ${res.status}`);  
const data = await res.json();
```

```
const usd = data['USDBRL']; // chave une o par sem traço
```

```
console.log(`Compra: R$ ${(+usd.bid).toFixed(2)}`);
```

```
console.log(`Venda: R$ ${(+usd.ask).toFixed(2)}`);
```

```
console.log(`Variação: ${usd.pctChange}%`);
```

```
// 2) Paralelo USD + EUR:  
const [rUSD, rEUR] = await Promise.all(['USD-BRL', 'EUR-BRL'].map(p => fetch(`${BASE}/last/${p}`).then(r => r.json())));
```



Correção

```
const BASE = 'https://api.open-meteo.com/v1/forecast';
```

```
async function buscarClima(lat, lon) {  
  const url = `${BASE}?latitude=${lat}&longitude=${lon}
```

```
    &current=temperature_2m,windspeed_10m`;
```

```
  const res = await fetch(url);  
  if (!res.ok) throw new Error(`Erro ${res.status}`);  
  return (await res.json()).current;
```

```
}
```

```
// 4) 3 cidades do RS em paralelo:
```

```
const cidades = [{ nome: 'Porto Alegre', lat: -30.03, lon: -51.23 },  
  { nome: 'Caxias do Sul', lat: -29.17, lon: -51.18 }]
```



Correção

```
const BASE = 'https://brasilapi.com.br/api/cnpj/v1';
```

```
async function buscarEmpresa(cnpj) {  
  const limpo = cnpj.replace(/[\.\-\/]/g, '').trim();
```

```
  const res = await fetch(`${BASE}/${limpo}`);
```

```
  if (!res.ok) throw new Error(`CNPJ inválido ou não encontrado (${res.status})`);
```

```
  const d = await res.json();  
  return { razao: d.razao_social, cidade: `${d.municipio}/${d.uf}`,
```

```
    situacao: d.descricao_situacao_cadastral,  
    atividade: d.cnae_fiscal_descricao };
```

```
}
```

```
// 5) Verificar situação cadastral:  
const emp = await buscarEmpresa('00000000000191');  
if (emp.situacao !== 'ATIVA')
```



Pesquisa Individual

Consumo de APIs REST em JavaScript

Leituras recomendadas:

- MDN fetch: developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API/Using_Fetch
- JSONPlaceholder: jsonplaceholder.typicode.com | ViaCEP: viacep.com.br

Perguntas de pesquisa (10–15 min)

- 1) Por que `fetch()` não rejeita em erros 4xx/5xx? Como tratar corretamente?
(Dica: cheque `res.ok` – true apenas para status 200–299)
- 2) O que é CORS? Por que o browser bloqueia requisições entre domínios diferentes?
- 3) Qual a diferença prática entre GET, POST, PUT, PATCH e DELETE?
- 4) O que significa uma API ser "stateless"? Por que isso importa?
- 5) Bônus: pesquise `AbortController` – como cancelar um fetch que está demorando?



Próxima aula

Aula 11

Métodos funcionais de arrays: map, filter e reduce – transformar os dados que você acabou de buscar.

Para chegar bem:

- Traga dúvidas sobre fetch e as APIs dos exercícios
- Complete os 3 exercícios (as APIs são gratuitas e sem cadastro!)
- Pesquise o que é CORS e como o servidor o configura



Obrigado